

Session 2

Computing, Robotics and Bionics

Sergio Sama Rubio

Sant Cugat del Vallès, 10-Sep-2026

Announcements

Previously on CRB1

- Session 1: course intro, grading criteria
- Google Drive, Github, Google Colab

Questions?

Today's Agenda

- Refresher Module 1 (PDF): Python, NumPy and the scientific stack
- Notebook 1 — Python for Bioengineers
 - data model, containers, control flow, functions, errors
- Notebook 2 — NumPy for Bioengineers
 - arrays, indexing, broadcasting, reductions, vectorization
- Kahoot

The Scientific Python Stack

Where today's session fits in the ML workflow

- NumPy — the numerical array; the unit of scientific computing in Python (today)
 - Pandas — labelled tables for loading and cleaning data (Session 3)
 - Matplotlib — plotting and visualisation (Session 3)
 - Scikit-Learn — classical ML models, one uniform interface: fit / predict / transform
 - PyTorch — tensors, autograd and neural networks (deep-learning half of the course)
-
- The ML workflow: Get data → Explore → Prepare → Train → Evaluate → iterate

Python's Data Model

Notebook 1 — Python for Bioengineers

- Objects & names: a variable is a name bound to an object, not a box
 - mutable objects (list, dict, set) can be aliased — copy on purpose
- Four containers: list, tuple, dict, set
- Indexing & slicing: `seq[start:stop:step]` — the same syntax reused in NumPy
- Control flow, comprehensions, and functions as first-class objects
- Errors & exceptions: `try / except` so one bad record doesn't crash a pipeline
- A first look at classes: reading `model.fit(X, y)` and `model.coef_`

Let's practice — Notebook 1

https://github.com/SergiSama/UIC-CRB1-2026-2027/blob/main/m1_python_basics_student.ipynb

[Open Notebook 1 in Google Colab](#) →

What is a Vector

https://www.youtube.com/watch?v=fNk_zzaMoSs

Numpy

Library to efficiently handle multi-dimensional arrays

- Efficient numerical computing:
- Internal structure: fixed-type, multidimensional, contiguous memory
- Element-wise operations, broadcasting
- Indexing, slicing, reshaping
- Linear algebra, statistics, random sampling
- Vectorization: avoid Python loops, use fast C-based operations
- Benefits: performance, cleaner code, scalability

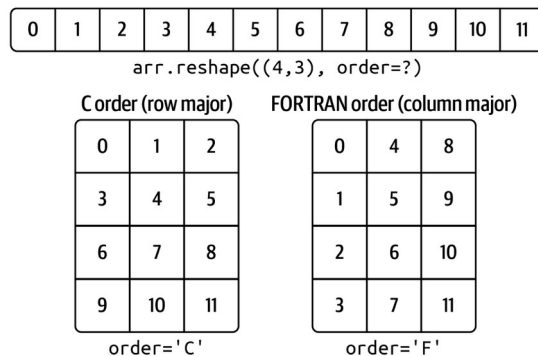
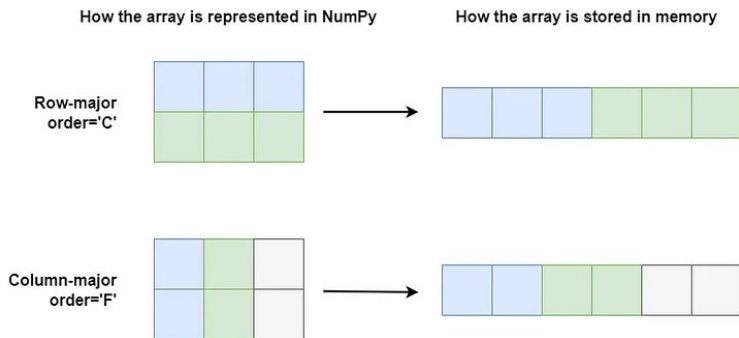
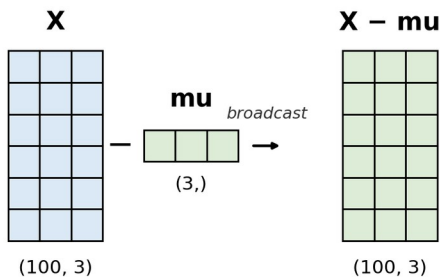


Figure A.3: Reshaping in C (row major) or FORTRAN (column major) order

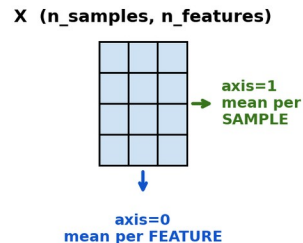
NumPy: Broadcasting & Axes

Combining shapes and reducing arrays without writing a single loop

- Boolean masks & fancy indexing select without an if
- Broadcasting combines different but compatible shapes — no loops, no copies
- Reductions run along an axis: the axis you name is the axis that disappears
- Views vs copies: a slice is usually a view — copy on purpose with `.copy()`



The length-1 axis of **mu** is stretched to match every one of X's 100 rows — no loop, no copy of **mu**.



"The axis you name is the axis that disappears."

Let's practice — Notebook 2

https://github.com/SergiSama/UIC-CRB1-2026-2027/blob/main/m1_numpy_student.ipynb

[Open Notebook 2 in Google Colab](#) →

Before Next Session

- Finish both notebooks' exercises (solution cells included — check your own work)
- Re-read Refresher Module 1 (PDF) sections 1-3 if you haven't already
- Session 3 preview: Pandas & Matplotlib

Q & A



End of Session 2

Computing, Robotics and Bionics

Sergio Sama Rubio

Sant Cugat del Vallès, 10-Sep-2026